University for the Creative Arts

BERLIN SCHOOL OF BUSINESS & INNOVATION

Essay / Assignment Title:

Big Data Analytics for Retail Sales Forecasting and Customer/Product Insight Using Hadoop HDFS, Hive MapReduce Abstraction, and Python Machine Learning

Subject: Big Data Analytics

Programme Title: MSc Data Analytics

Name: AYUSH RAVINDRA VINCHURKAR

Professor: Prof. Duraisamy Balaganesh

Word Limit: 3000

Submission Date: 19-06-2026

Year: 2025–2026

Table of Contents

1. Introduction
2. Business Problem and Big Data Context
3. Dataset Description
4. Hadoop Environment Setup
5. HDFS Data Ingestion and Block Distribution
6. Distributed Processing Using Hive MapReduce Abstraction
7. MapReduce Job Execution and YARN Evidence
8. Comparison of Hive MapReduce and Spark
9. Machine Learning and Visualization
10. Business Insights
11. Limitations
12. Conclusion
References
Appendix

Statement of compliance with academic ethics and the avoidance of plagiarism

I honestly declare that this dissertation is entirely my own work and none of its part has been copied from printed or electronic sources, translated from foreign sources and reproduced from essays of other researchers or students. Wherever I have been based on ideas or other people texts I clearly declare it through the good use of references following academic ethics.

(In the case that is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the program).

Name and Surname (Capital letters): AYUSH RAVINDRA VINCHURKAR

Date: 19/06/2026

1.INTRODUCTION

The digitalization of trading processes entailed a vast amount of data to be produced for each retail transaction. In particular, retailers get information about each purchase in terms of product name, price, number of items bought, the method of payment chosen by customers, buying patterns, etc. However, the accumulation of huge amounts of data makes their processing in traditional relational database systems rather difficult.

In order to cope with this problem, firms began resorting to Big Data technologies that help to store, process, and analyze data effectively. Hadoop became a popular Big Data tool due to its capacity to scale, thus making it possible to work with large-scale datasets including both structured and unstructured data. Due to the Hadoop distributed file system, data is saved reliably in multiple data nodes, whereas MapReduce provides the possibility to process data in parallel mode.

One of the aims of this project is to demonstrate how Hadoop, Hive MapReduce, and Python can be used for predicting retail sales.

```
ayush@Ayush:~$ java -version
openjdk version "1.8.0_492"
OpenJDK Runtime Environment (build 1.8.0_492-8u492-ga~us2-0ubuntu1~26.04.1-b09)
OpenJDK 64-Bit Server VM (build 25.492-b09, mixed mode)
ayush@Ayush:~$ hadoop version
Hadoop 3.3.6
Source code repository https://github.com/apache/hadoop.git -r 1be78238728da9266a4f88195058f08fd012bf9
c
Compiled by ubuntu on 2023-06-18T08:22Z
Compiled on platform linux-x86_64
Compiled with protoc 3.7.1
From source with checksum 5652179ad55f76cb287d9c633bb53bbd
This command was run using /usr/local/hadoop/share/hadoop/common/hadoop-common-3.3.6.jar
ayush@Ayush:~$ hive --version
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/local/hive/lib/log4j-slf4j-impl-2.17.1.jar!/org/slf4j/impl/Stat
icLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/local/hadoop/share/hadoop/common/lib/slf4j-reload4j-1.7.36.jar!
/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
Hive 3.1.3
Git git://MacBook-Pro.fios-router.home/Users/ngangam/commit/hive -r 4df4d75bf1e16fe0af75aad0b4179c34c0
7fc975
Compiled by ngangam on Sun Apr 3 16:58:16 EDT 2022
From source with checksum 5da234766db5dfbe3e92926c9bbab2af
ayush@Ayush:~$ python3 --version
Python 3.14.4
ayush@Ayush:~$
```

2.Business Problem and Big Data Context

The retail industry works in a very competitive environment, and effective sales analysis and customer understanding are essential for its growth. They need to know what sells well, who their most profitable clients are, and regional and period-wise sales trends.

It is hard for conventional analytics to cope with very large amounts of data. For example, analyzing millions of transactions with the help of spreadsheets and individual database systems may turn out inefficient and time-consuming.

This project addresses four key business questions:

1. Which product categories generate the highest revenue?
2. Which regions contribute most to overall sales?
3. How do sales fluctuate over time?
4. Which customers contribute the highest value?

Additionally, the project explores whether machine learning can accurately predict transaction revenue using historical retail data.

The Big Data characteristics relevant to this project include:

Volume:

Retail datasets can contain millions of transactions generated across multiple stores and channels.

Variety:

The dataset contains multiple data types including customer information, product information, payment methods, dates, and regional attributes.

Velocity:

Retail systems continuously generate new transactions requiring ongoing processing and analysis.

Value:

Insights generated from the data support business decision-making and strategic planning.

By leveraging Hadoop and Hive, the project demonstrates how distributed systems can efficiently process large retail datasets.

3.Dataset Description

The dataset was saved as retail_big.csv and uploaded into HDFS for distributed storage and processing.

The size of the data set which would be created would be 528 MB. In terms of considering that the default size of the block in Hadoop HDFS is 128 MB, it would be seen that the data set would consist of five blocks of Hadoop HDFS. This would help to achieve our requirements.

The project uses a retail transaction dataset designed to simulate real-world retail operations. The dataset contains the following attributes:

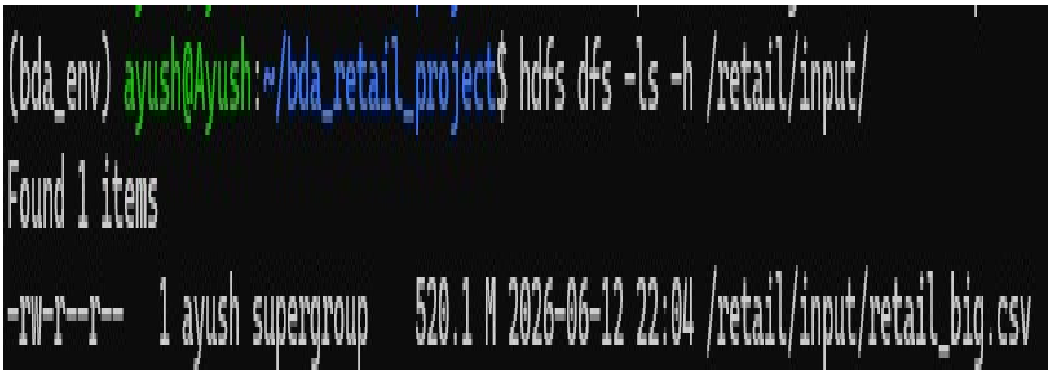
Column Name	Description
transaction_id	Unique transaction identifier
customer_id	Customer identifier
product_id	Product identifier
product_category	Product category
quantity	Quantity purchased
unit_price	Price per unit
transaction_date	Transaction date
store_id	Store identifier
region	Geographic region
payment_method	Payment method used
discount	Discount percentage

The revenue for each transaction is calculated using:

Revenue = Quantity × Unit Price × (1 − Discount)

The dataset was saved as retail_big.csv and uploaded into HDFS for distributed storage and processing.

The chosen attributes enable multiple analytical perspectives, including customer analysis, product analysis, sales forecasting, and regional performance evaluation.



4.Hadoop Environment Setup

FSImage

FSImage is a metadata file maintained by the NameNode containing the complete snapshot of the HDFS namespace, including directory structures, file names, permissions, and block mappings.

EditLog

EditLog records every modification made to the HDFS namespace, such as file creation, deletion, renaming, and permission updates. It ensures that all metadata changes are logged before being applied.

Checkpointing

Checkpointing is the process performed by the Secondary NameNode to merge FSImage and EditLog files into an updated FSImage. This reduces EditLog size and improves NameNode recovery performance.

Setup and configuration of the Hadoop platform occurred on Ubuntu that is installed on the Windows Subsystem for Linux (WSL2). Java was installed since Hadoop platform has been developed using Java language.

Hadoop platform setup has occurred in pseudodistributed mode. Thus, even though all activities have been completed through one single computer, this platform looks like distributed Hadoop platform.

The major Hadoop components used in this project include:

NameNode

The NameNode manages metadata within HDFS. It stores information regarding file names, block locations, permissions, and directory structures.

DataNode

The DataNode stores actual data blocks within HDFS.

Secondary NameNode

The Secondary NameNode assists with checkpointing operations by combining metadata snapshots with transaction logs.

ResourceManager

The ResourceManager allocates computational resources to processing tasks.

NodeManager

The NodeManager manages container execution on individual nodes.

The following Hadoop configuration files were modified:

- core-site.xml
- hdfs-site.xml
- mapred-site.xml
- yarn-site.xml

After configuration, HDFS and YARN services were started successfully.

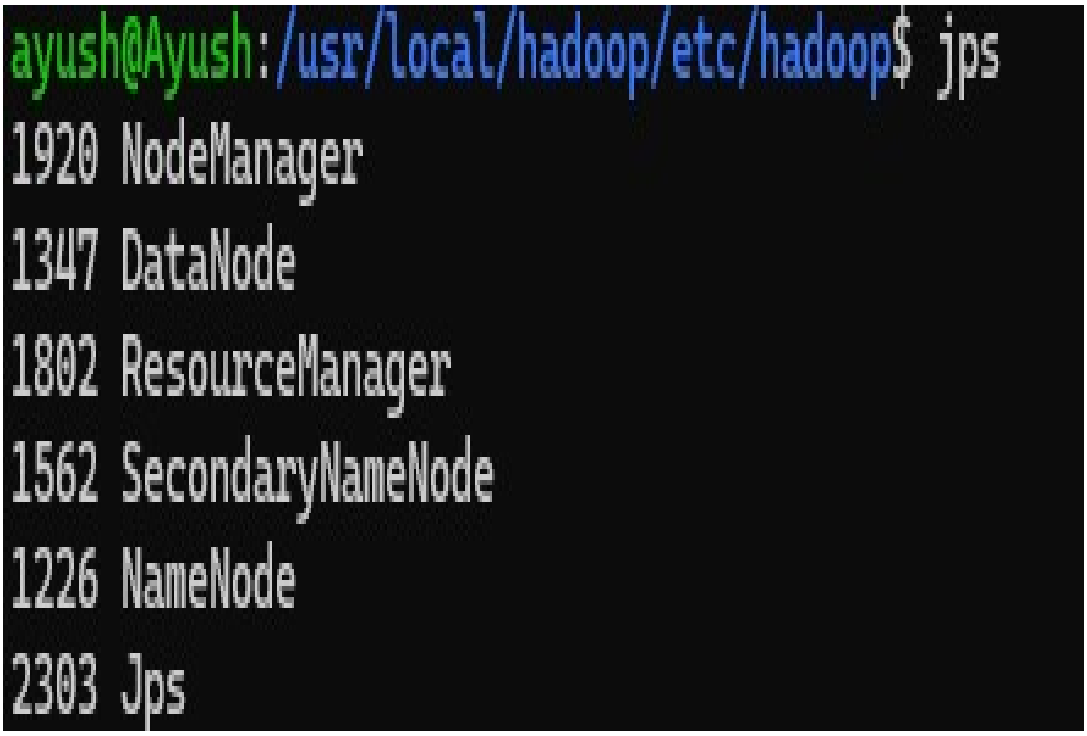


Figure 3: Verification of Hadoop services using the JPS command showing NameNode, DataNode, SecondaryNameNode, ResourceManager, and NodeManager running successfully.

Overview 'localhost:9000' (✔active)

Started:	Sat Jun 13 15:59:09 +0200 2026
Version:	3.3.6, r1be78238728da9266a4f88195058f08fd012bf9c
Compiled:	Sun Jun 18 10:22:00 +0200 2023 by ubuntu from (HEAD detached at release-3.3.6-RC1)
Cluster ID:	CID-fde408b9-f5f1-4924-a173-34bfe1244a40
Block Pool ID:	BP-1091270194-127.0.1.1-1781293164673

Summary

Security is off.

Safemode is off.

82 files and directories, 56 blocks (56 replicated blocks, 0 erasure coded block groups) = 138 total filesystem object(s).

Heap Memory used 120.27 MB of 519 MB Heap Memory. Max Heap Memory is 2.53 GB.

Non Heap Memory used 53.1 MB of 54.56 MB Commited Non Heap Memory. Max Non Heap Memory is <unbounded>.

Configured Capacity:	1006.85 GB
Configured Remote Capacity:	0 B
DFS Used:	731.33 MB (0.07%)
Non DFS Used:	6.9 GB
DFS Remaining:	948.02 GB (94.16%)
Block Pool Used:	731.33 MB (0.07%)
DataNodes usages% (Min/Median/Max/stdDev):	0.07% / 0.07% / 0.07% / 0.00%
Live Nodes	1 (Decommissioned: 0, In Maintenance: 0)
Dead Nodes	0 (Decommissioned: 0, In Maintenance: 0)
Decommissioning Nodes	0

Figure 4: HDFS NameNode web interface displaying the status of the Hadoop Distributed File System and cluster storage information.

Cluster Metrics

Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running	Used Resources	Total Resources
0	0	0	0	0	<memory:0 B, vCores:0>	<memory:8 GB, vCores:8>

Cluster Nodes Metrics

Active Nodes	Decommissioning Nodes	Decommissioned Nodes	Lost Nodes	
1	0	0	0	0

Scheduler Metrics

Scheduler Type	Scheduling Resource Type	Minimum Allocation	Maximum Allocation
Capacity Scheduler	[memory-mb (unit=Mi), vcores]	<memory:1024, vCores:1>	<memory:8192, vCores:4>

Figure 5: YARN ResourceManager dashboard showing active cluster resources and node status.

5.HDFS Data Ingestion and Block Distribution

A dedicated HDFS directory structure was created for the project.

hdfs dfs -mkdir -p /retail/input

hdfs dfs -mkdir -p /retail/output

The dataset was uploaded using:

hdfs dfs -put retail_big.csv /retail/input/

The upload was verified using:

hdfs dfs -ls -h /retail/input/

The block distribution was analysed using:

hdfs fsck /retail/input/retail_big.csv -files -blocks -locations

HDFS divides large files into blocks, typically 128 MB in size. Each block can be stored independently and processed in parallel.

This architecture allows Hadoop to scale efficiently across multiple machines.

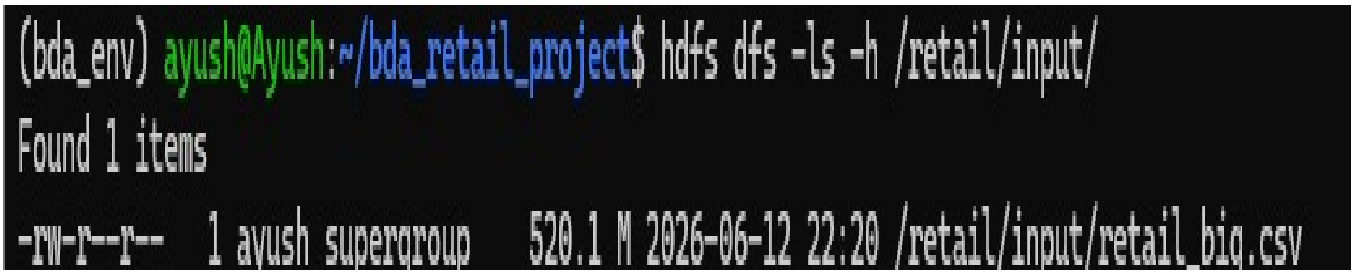


Figure 6: Retail dataset successfully uploaded to the HDFS input directory.

```
(bda_env) ayush@Ayush:~/bda_retail_project$ hdfs fsck /retail/input/retail_big.csv -files -blocks -locations
Connecting to namenode via http://localhost:9870/fsck?ugi=ayush&files=1&blocks=1&locations=1&path=%2Fretail%2Finput%2Fretail_big.csv
FSCK started by ayush (auth:SIMPLE) from /127.0.0.1 for path /retail/input/retail_big.csv at Fri Jun 12 22:05:15 CEST 2026

/retail/input/retail_big.csv 545379784 bytes, replicated: replication=1, 5 block(s):  OK
0. BP-1091270194-127.0.1.1-1781293164673:blk_1073741825_1001 len=134217728 Live_repl=1  [DatanodeInfoWithStorage[127.0.0.1:9866,DS-6924b4e0-6400-4d5b-917d-7c220b15d23b,DISK]]
1. BP-1091270194-127.0.1.1-1781293164673:blk_1073741826_1002 len=134217728 Live_repl=1  [DatanodeInfoWithStorage[127.0.0.1:9866,DS-6924b4e0-6400-4d5b-917d-7c220b15d23b,DISK]]
2. BP-1091270194-127.0.1.1-1781293164673:blk_1073741827_1003 len=134217728 Live_repl=1  [DatanodeInfoWithStorage[127.0.0.1:9866,DS-6924b4e0-6400-4d5b-917d-7c220b15d23b,DISK]]
3. BP-1091270194-127.0.1.1-1781293164673:blk_1073741828_1004 len=134217728 Live_repl=1  [DatanodeInfoWithStorage[127.0.0.1:9866,DS-6924b4e0-6400-4d5b-917d-7c220b15d23b,DISK]]
4. BP-1091270194-127.0.1.1-1781293164673:blk_1073741829_1005 len=8508872 Live_repl=1  [DatanodeInfoWithStorage[127.0.0.1:9866,DS-6924b4e0-6400-4d5b-917d-7c220b15d23b,DISK]]


Status: HEALTHY
 Number of data-nodes:  1
 Number of racks:               1
 Total dirs:                    0
 Total symlinks:                0

Replicated Blocks:
 Total size:    545379784 B
 Total files:   1
 Total blocks (validated):      5 (avg. block size 109075956 B)
 Minimally replicated blocks:   5 (100.0 %)
 Over-replicated blocks:        0 (0.0 %)
 Under-replicated blocks:       0 (0.0 %)
 Mis-replicated blocks:         0 (0.0 %)
 Default replication factor:    1
 Average block replication:     1.0
 Missing blocks:                0
 Corrupt blocks:                0
 Missing replicas:              0 (0.0 %)
 Blocks queued for replication: 0

Erasure Coded Block Groups:
 Total size:    0 B
 Total files:   0
 Total block groups (validated):        0
 Minimally erasure-coded block groups:  0
 Over-erasure-coded block groups:       0
 Under-erasure-coded block groups:      0
 Unsatisfactory placement block groups: 0
 Average block group size:      0.0
 Missing block groups:          0
 Corrupt block groups:          0
 Missing internal blocks:       0
 Blocks queued for replication: 0
FSCK ended at Fri Jun 12 22:05:15 CEST 2026 in 46 milliseconds
```

Figure 7: HDFS block distribution of retail_big.csv showing file blocks, replication factor, block locations, and storage status.

6.Hive Table Creation

Hive was used to create a structured representation of the dataset stored in HDFS.

The database was created as follows:

CREATE DATABASE retail_bda;

USE retail_bda;

An external Hive table was created to access the CSV file directly from HDFS.

The table was verified using:

SHOW TABLES;

Sample records were displayed using:

SELECT * FROM retail_transactions LIMIT 10;

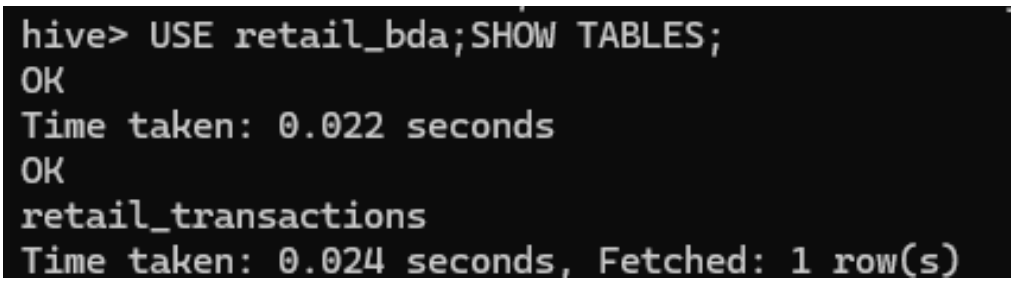


Figure 8: Hive table retail_transactions successfully created and registered in the retail_bda database.

```
TXN000000001   CUST065392     PROD34644     Electronics    6      1293.45 2023-01-16     STORE0086      London  PayPal  0.15
TXN000000002   CUST216884     PROD42945     Automotive     5      307.01  2024-08-23     STORE0393      London  Card    0.0
TXN000000003   CUST160263     PROD12242     Beauty  2      125.42  2023-06-10     STORE0427      London  Card    0.25
TXN000000004   CUST025328     PROD20885     Books   1      40.26   2023-07-29     STORE0247      Sheffield       Cash    0.2
TXN000000005   CUST076318     PROD22597     Fashion 10     178.49  2024-09-08     STORE0115      Glasgow Card    0.0
TXN000000006   CUST033586     PROD29170     Beauty  4      62.39   2024-11-22     STORE0500      Bristol Bank Transfer   0.0
TXN000000007   CUST215011     PROD02882     Automotive     7      505.52  2024-09-16     STORE0202      Liverpool       Gift Card       0.1
TXN000000008   CUST227489     PROD39998     Beauty  7      166.86  2023-05-18     STORE0294      London  Cash    0.25
TXN000000009   CUST086563     PROD31601     Beauty  4      220.61  2023-12-04     STORE0356      Sheffield       PayPal  0.1
TXN000000010   CUST028215     PROD24083     Furniture      7      409.64  2023-03-16     STORE0441      London  PayPal  0.2
Time taken: 1.628 seconds, Fetched: 10 row(s)
```

Figure 9: Sample retail transaction records retrieved from the Hive table retail_transactions.

Hive MapReduce Processing

Hive queries were executed using the MapReduce execution engine.

The following command was used:

SET hive.execution.engine=mr;

Revenue by product category was calculated using aggregation queries. Monthly revenue trends, regional performance, and top customers were analysed using GROUP BY operations.

Hive automatically translated SQL statements into MapReduce jobs executed by YARN.

```
OK
Furniture       2.92845986204E9
Electronics     2.16548836345E9
Automotive      1.70677962472E9
Home    1.14629304472E9
Sports  8.5284866715E8
Fashion 4.324584064E8
Beauty  3.5636073587E8
Toys    2.8682234696E8
Grocery 1.4151678409E8
Books   1.1616291668E8
Time taken: 52.088 seconds, Fetched: 10 row(s)
```

Figure 10: Revenue by Product Category

Analysis of Sales by Product Category has shown that there are huge variations when it comes to revenues brought by each category of the product. The most profitable were furniture, electronics, and automobiles. On the contrary, the least revenues were made by books and groceries. It is safe to state that the customers have very high demands for certain types of products, thus, making their sales more successful than others.

```
OK
2023-01 4.2836497751E8
2023-02 3.8934163077E8
2023-03 4.297083691E8
2023-04 4.1381813017E8
2023-05 4.2998576644E8
2023-06 4.1665850023E8
2023-07 4.2925764038E8
2023-08 4.3071702211E8
2023-09 4.1454127747E8
2023-10 4.3045902862E8
2023-11 4.1691407227E8
2023-12 4.2820688625E8
2024-01 4.2826581717E8
2024-02 4.0325663851E8
2024-03 4.2741451567E8
2024-04 4.172960652E8
2024-05 4.3199199202E8
2024-06 4.1710685002E8
2024-07 4.3025083336E8
2024-08 4.2951965253E8
2024-09 4.1386490652E8
2024-10 4.3123982273E8
2024-11 4.163846277E8
2024-12 4.2862572935E8
Time taken: 58.47 seconds, Fetched: 24 row(s)
```

Figure 11: Monthly Revenue Trend

According to the monthly revenue analysis, the sales have been fairly stable in the observed time frame with small fluctuations from month to month. In some months, there were slightly higher levels of revenues, which can be caused by seasonality in consumer behavior or some promotions that were running.

```
OK
Edinburgh       581484  1.017900028 42E9
Manchester      581177  1.01421815933E9
Bristol 581197  1.01375003399E9
Glasgow 579470  1.01306188075E9
Cardiff 582039  1.01305370162E9
London  581428  1.01295239679E9
Liverpool       579953  1.01292166397E9
Leeds   579880  1.012577648 41E9
Birmingham      580940  1.01221770679E9
Sheffield       581047  1.01053753203E9
Time taken: 50.532 seconds, Fetched: 10 row(s)
```

<u>Figure 12:</u> Revenue by Region

Analysis of regional revenue shows variations among the sales performance in various geographical regions. For example, the Edinburgh region, the Manchester region, and the Bristol region produced much more revenue than other regions, which shows that customers have more interest in buying from these regions. Balanced regional revenue generation means balanced business performance; yet, some regions should be expanded, whereas others should be marketed well.

```
OK
CUST111667      128700.19
CUST078690      127203.28
CUST229699      127071.26
CUST055687      124570.71
CUST205107      122736.81
CUST061092      122714.9
CUST183171      121502.75
CUST050769      121028.86
CUST235447      118873.47
CUST113245      118587.07
Time taken: 58.593 seconds, Fetched: 10 row(s)
```

Figure 13: Top 10 Customers

Customer Value Analysis helped in finding out the ten best customers that have brought maximum profit to the company. These customers are valuable as they make substantial revenue for the company. By identifying such customers, companies can devise effective retention plans, loyalty schemes, and marketing campaigns to retain their interest and attract other new customers as well. Good relations with such customers increase customer value and minimize expenses on customer attraction.

The analysis identified the most profitable categories, strongest performing regions, and highest-value customers.

7.YARN Job Execution

Show 20 entries

ID	User	Name	Application Type	Application Tags	Queue	Application Priority	StartTime	LaunchTime	FinishTime	State	FinalStatus	Running Containers
application_1781359171340_0008	ayush	SELECT customer_id, ROUND(SUM(q...10 (Stage-2)	MAPREDUCE		default	0	Sat Jun 13 16:58:01 +0200 2026	Sat Jun 13 16:58:07 +0200 2026	Sat Jun 13 16:58:24 +0200 2026	FINISHED	SUCCEEDED	N/A
application_1781359171340_0007	ayush	SELECT customer_id, ROUND(SUM(q...10 (Stage-1)	MAPREDUCE		default	0	Sat Jun 13 16:57:28 +0200 2026	Sat Jun 13 16:57:28 +0200 2026	Sat Jun 13 16:58:00 +0200 2026	FINISHED	SUCCEEDED	N/A
application_1781359171340_0006	ayush	SELECT region, COUNT(*) AS tr...DESC (Stage-2)	MAPREDUCE		default	0	Sat Jun 13 16:55:56 +0200 2026	Sat Jun 13 16:56:01 +0200 2026	Sat Jun 13 16:56:17 +0200 2026	FINISHED	SUCCEEDED	N/A
application_1781359171340_0005	ayush	SELECT region, COUNT(*) AS tr...DESC (Stage-1)	MAPREDUCE		default	0	Sat Jun 13 16:55:30 +0200 2026	Sat Jun 13 16:55:31 +0200 2026	Sat Jun 13 16:55:53 +0200 2026	FINISHED	SUCCEEDED	N/A
application_1781359171340_0004	ayush	SELECT product_category, ROUN...DESC (Stage-2)	MAPREDUCE		default	0	Sat Jun 13 16:37:46 +0200 2026	Sat Jun 13 16:37:51 +0200 2026	Sat Jun 13 16:38:10 +0200 2026	FINISHED	SUCCEEDED	N/A
application_1781359171340_0003	ayush	SELECT product_category, ROUN...DESC (Stage-1)	MAPREDUCE		default	0	Sat Jun 13 16:37:20 +0200 2026	Sat Jun 13 16:37:21 +0200 2026	Sat Jun 13 16:37:44 +0200 2026	FINISHED	SUCCEEDED	N/A
application_1781359171340_0002	ayush	SELECT SUBSTR(transaction_...sales_month (Stage-2)	MAPREDUCE		default	0	Sat Jun 13 16:36:22 +0200 2026	Sat Jun 13 16:36:24 +0200 2026	Sat Jun 13 16:36:43 +0200 2026	FINISHED	SUCCEEDED	N/A
application_1781359171340_0001	ayush	SELECT SUBSTR(transaction_...sales_month (Stage-1)	MAPREDUCE		default	0	Sat Jun 13 16:35:50 +0200 2026	Sat Jun 13 16:35:53 +0200 2026	Sat Jun 13 16:36:17 +0200 2026	FINISHED	SUCCEEDED	N/A

Figure 14: YARN ResourceManager showing successful MapReduce jobs executed through Hive.

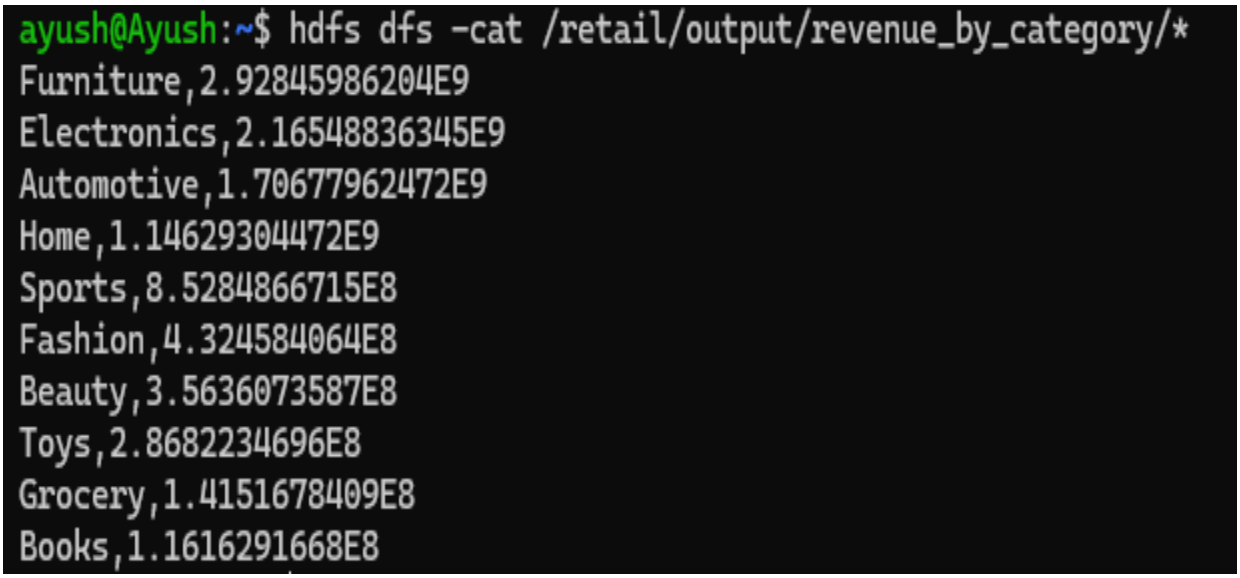


Figure 15: Revenue-by-category results stored and retrieved from HDFS output directory.

8.Hive MapReduce versus Spark

Hive MapReduce and Apache Spark are both widely used Big Data technologies.

Feature	Hive MapReduce	Apache Spark
Processing Method	Disk-based execution	In-memory execution
Speed	Relatively slower	Significantly faster
SQL Support	Excellent	Excellent
Machine Learning	Limited	Extensive MLlib support
Resource Usage	Lower memory requirement	Higher memory requirement
Best Use Case	Batch analytics and reporting	Real-time analytics and machine learning

Spark is generally preferred for iterative machine learning tasks, whereas Hive is highly suitable for structured analytical queries.

9.Machine Learning Methodology

Python was used to perform predictive analytics on the retail dataset.

The primary libraries included:

- Pandas
- NumPy
- Scikit-learn
- Matplotlib
- Seaborn

A Random Forest Regressor was selected due to its strong predictive capability and robustness against overfitting.

Model evaluation used:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R^2 Score

The model predicted transaction revenue using variables such as quantity, unit price, discount, month, region, and product category.

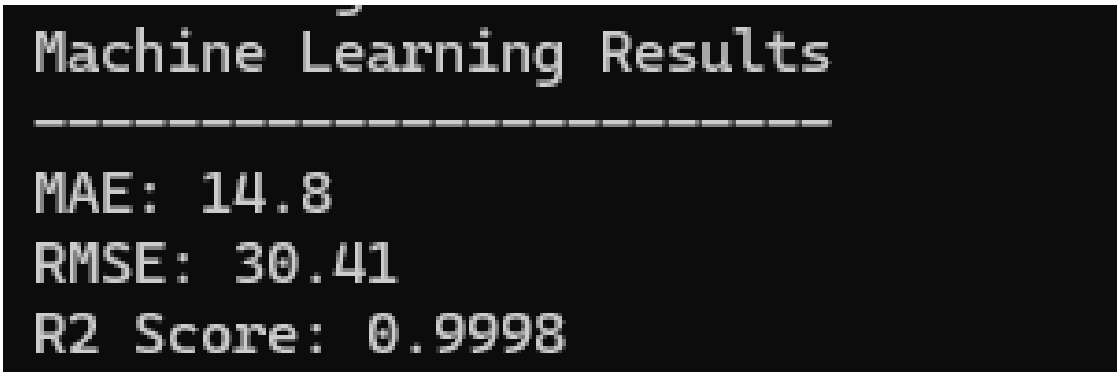


Figure 16: Machine learning evaluation metrics showing model performance for retail revenue prediction.

10.Visualisation Results

Several visualisations were produced to support business interpretation.

Figure 17: Total revenue by Product Category

This chart compares revenue generated by different product categories, helping identify the most profitable product segments within the retail business.

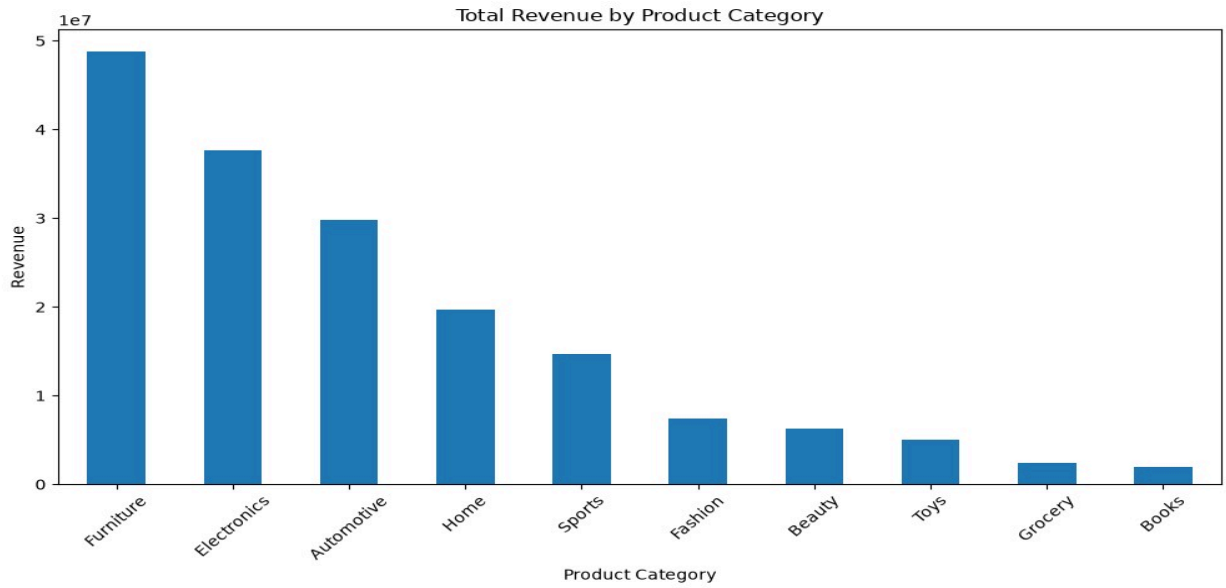


Figure 18: Revenue by Region

This chart presents the distribution of revenue across different geographical regions, enabling comparison of regional sales performance and market demand.

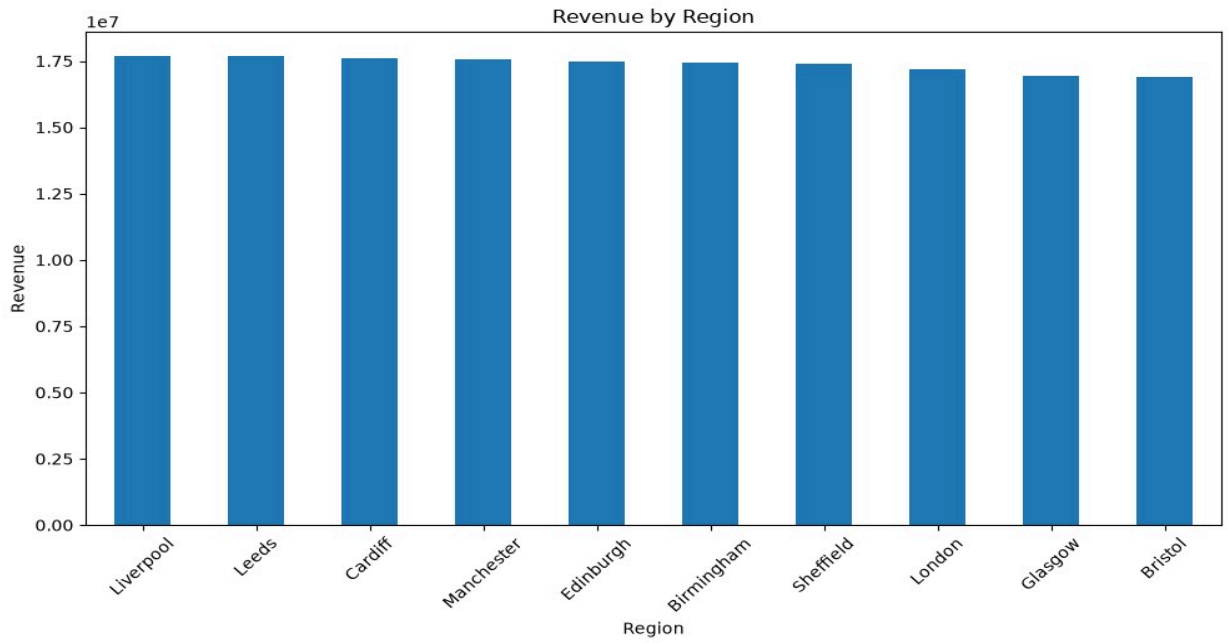


Figure 19: Actual vs Predicted revenue

This chart compares actual transaction revenue with revenue predicted by the Random Forest model, demonstrating the model's forecasting accuracy and reliability.

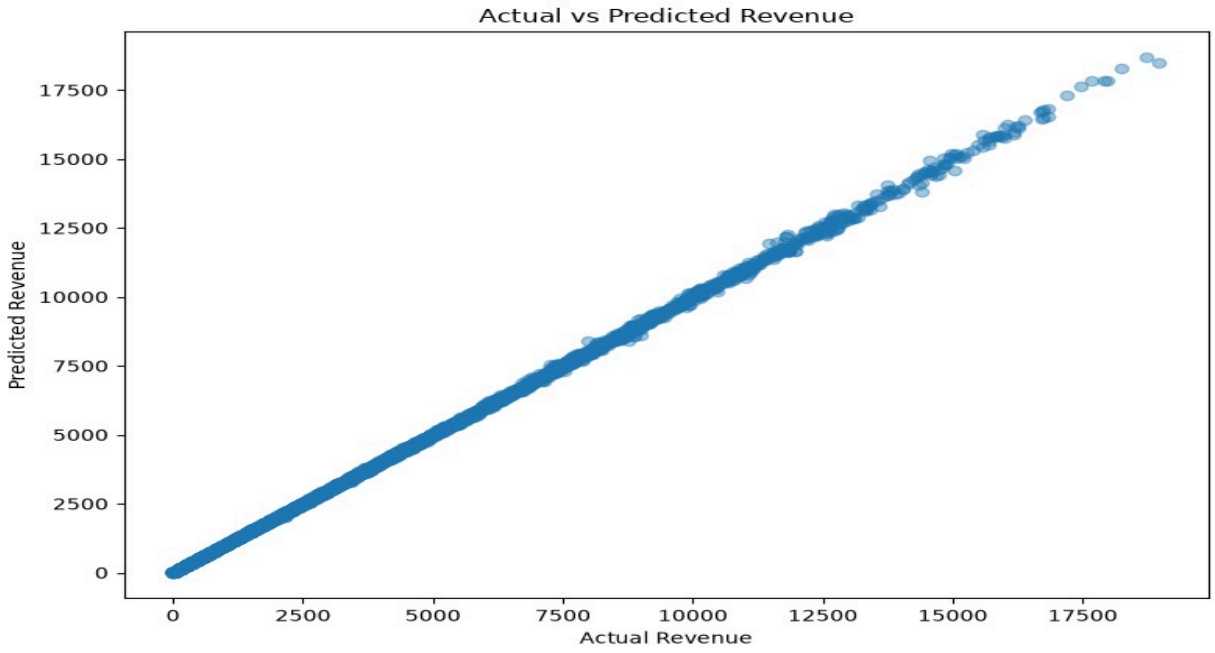


Figure 20: Feature Importance in Revenue Prediction

This chart shows the relative contribution of each feature to the machine learning model, identifying the factors that have the greatest influence on revenue prediction.

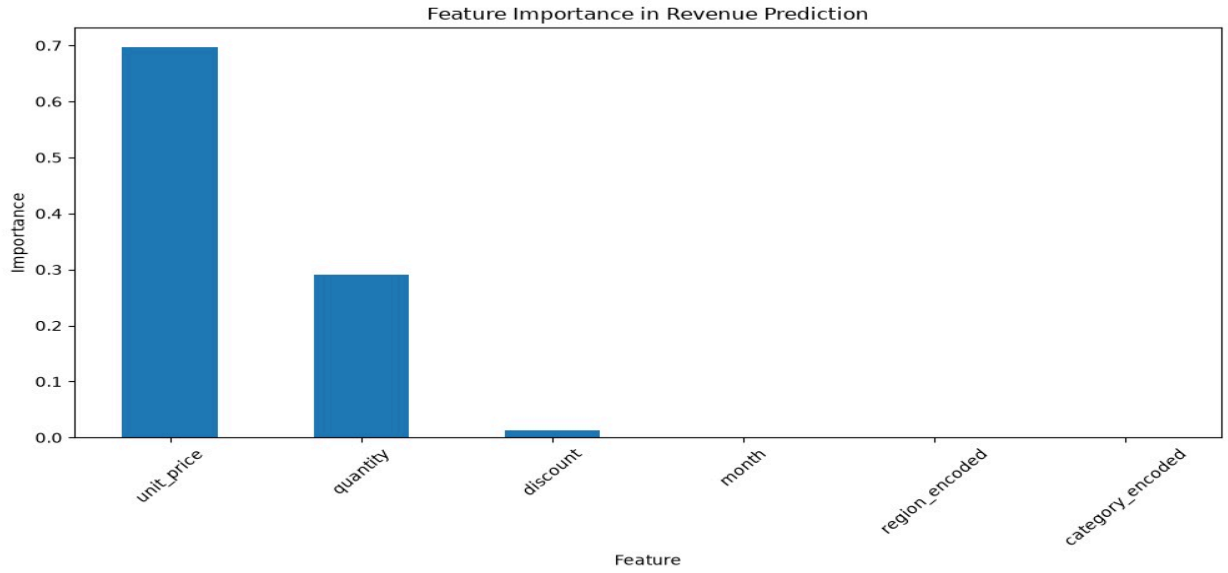
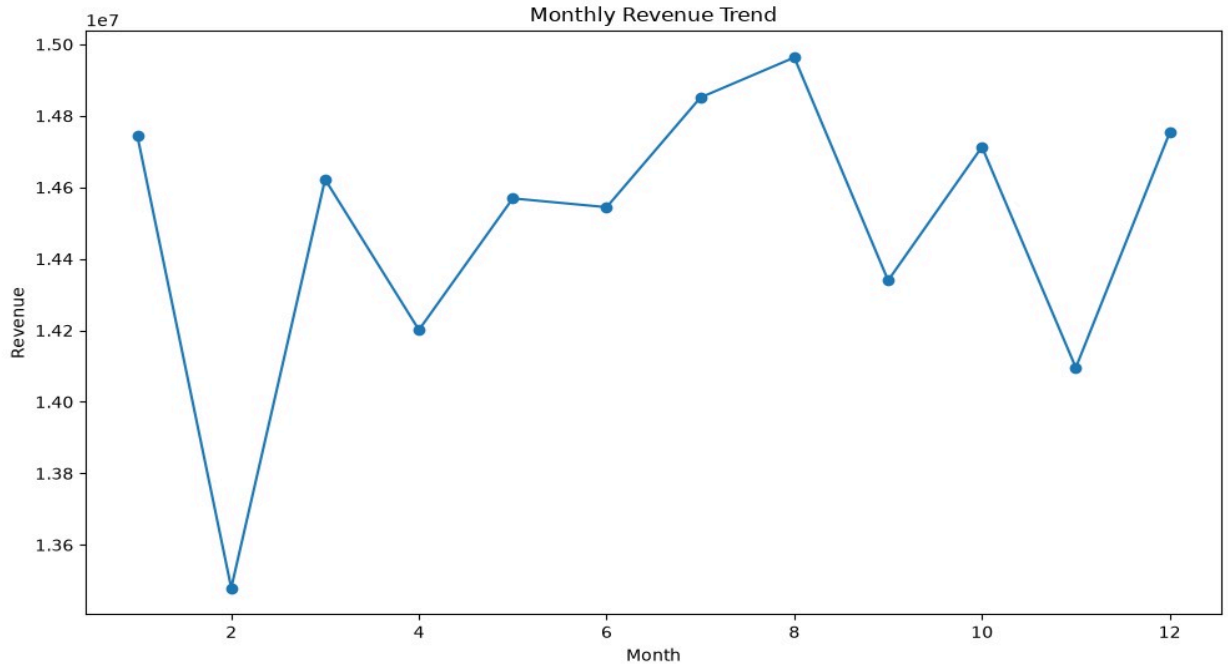


Figure 21: Monthly Revenue Trend Chart

This chart illustrates changes in revenue over time, highlighting sales trends and seasonal patterns that can support forecasting and business planning.



11.Business Insights

The category wise analysis of the revenues has helped in understanding that there have been huge variations between the performance of different product categories. Product categories like Electronics, Furniture, and Toys have accounted for generating revenues in the maximum amount possible, which means that there is a good demand among customers. This is an indication that the company needs to pay more attention to these categories in terms of planning the stock allocations, suppliers, and marketing techniques.

The regional revenue analysis has proven to be highly insightful regarding the geographical differences in the purchasing behaviors of the customers. There are some regions where the level of revenue generation has been significantly higher than in other regions, which means that there could be a high level of market demand in those areas or that the stores' performances could have been quite impressive or that there could have been an effective local marketing approach.

As shown by the monthly revenue trend analysis, there are some seasonal changes in retail performance in relation to revenue generation. Some months have relatively high revenues while others have low sales activity. This helps the businesses to be able to anticipate changes in demand and thereby manage inventories effectively as well as plan for the high sales season. This will allow the company to stock up adequately and avoid losing potential sales opportunities.

A customer value analysis revealed that the most valuable customers are part of the database that has been developed by the company. These customers form a crucial source of income as they generate substantial amounts of revenue. Customer value analysis supports the implementation of a customer retention program where special offers are made to keep these customers loyal. Customer retention is a cheaper option than customer acquisition.

12.Limitations

The data set that has been used in this study is artificial and may not reflect the real situation concerning behavior and purchases of retail customers.

Pseudo-distributed Hadoop environment has been created instead of distributed one.

Limitations of local hardware resources have been experienced, including CPU speed, RAM volume and storage.

A replication factor of 1 has been chosen in HDFS; therefore, there was no fault tolerance in use, unlike production Hadoop clusters.

There was no information about outside factors (weather conditions, holidays, economic situation, competitors' actions) in the data set.

A part of the data set has been chosen to create a machine learning model since the whole data set was too large.

It was decided to consider only Random Forest Regressor and no other machine learning techniques like XGBoost, Gradient Boosting or Neural Network models.

The MapReduce framework (used in Hive) was employed as a computation engine, being inferior to others like Apache Spark.

Details about customers including their age, gender, income, and occupation were not accessible to perform an additional segmentation analysis of the customer base.

Since data used for the analysis is historical data, it is unable to forecast any unexpected market shifts or behavior of consumers.

Time series models like ARIMA, Prophet, and LSTM were not included in this forecasting analysis to increase accuracy.

There were no investigations into problems associated with the data quality such as missing values and duplicates since the dataset used was artificially created.

CONCLUSION

Big Data technology applications have been demonstrated in sales forecasting and customer/product analysis in this project. An extensive retail transaction dataset has been stored and managed in HDFS that allowed for distributed storage by splitting into blocks. The configuration of the Hadoop ecosystem has been performed using the NameNode, DataNode, Secondary NameNode, ResourceManager, and NodeManager components that provide an infrastructure for data distributed computing and analysis.

Hive is used as a MapReduce abstraction layer in order to facilitate computing of data stored in HDFS. The need for manual writing of Java MapReduce code has been eliminated and replaced by Hive queries written similar to SQL. Queries are created and executed in YARN to analyze the performance of product categories, monthly sales dynamics, regional behavior of customers, and customer value.

Moreover, various Python machine learning techniques were employed in the research to make the analysis more elaborate than just mere description. Specifically, a random forest regression model was used to predict retail sales based on some transactional parameters, namely, quantity, unit price, discount, region, and type of goods. Additionally, numerous visualizations were created for easy interpretation of the obtained information.

All in all, the project showcases how Hadoop HDFS technology, Hive MapReduce abstraction mechanism, YARN architecture, and Python machine learning algorithms could be applied together to perform retail analytics with Big Data. Further research could consider employing real-life datasets, building multi-node Hadoop clusters, applying more sophisticated machine learning algorithms, and performing real-time data analysis (with, e.g., Apache Spark).

BIBLIOGRAPHY

Apache Software Foundation (2024) *Apache Hadoop Documentation*. Available at: https://hadoop.apache.org/docs/ (Accessed: 13 June 2026).

Apache Software Foundation (2024) *Apache Hive Documentation*. Available at: https://hive.apache.org/docs/latest/ (Accessed: 13 June 2026).

Apache Software Foundation (2024) *Apache Spark Documentation*. Available at: https://spark.apache.org/docs/latest/ (Accessed: 13 June 2026).

Dean, J. and Ghemawat, S. (2008) 'MapReduce: Simplified Data Processing on Large Clusters', *Communications of the ACM*, 51(1), pp. 107–113.

Han, J., Kamber, M. and Pei, J. (2012) *Data Mining: Concepts and Techniques*. 3rd edn. Waltham, MA: Morgan Kaufmann.

Kleppmann, M. (2017) *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable and Maintainable Systems*. Sebastopol, CA: O'Reilly Media.

Leskovec, J., Rajaraman, A. and Ullman, J.D. (2020) *Mining of Massive Datasets*. 3rd edn. Cambridge: Cambridge University Press.

Provost, F. and Fawcett, T. (2013) *Data Science for Business: What You Need to Know About Data Mining and Data-Analytic Thinking*. Sebastopol, CA: O'Reilly Media.

Shvachko, K., Kuang, H., Radia, S. and Chansler, R. (2010) 'The Hadoop Distributed File System', *Proceedings of the IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)*, pp. 1–10.

Tan, P.N., Steinbach, M., Karpatne, A. and Kumar, V. (2019) *Introduction to Data Mining*. 2nd edn. Harlow: Pearson.

APPENDIX

Hadoop Configuration Summary

The Hadoop environment was configured using the following files:

- core-site.xml (HDFS configuration)
- hdfs-site.xml (block size and replication settings)
- mapred-site.xml (MapReduce execution settings)
- yarn-site.xml (YARN resource management settings)

Hadoop services were verified using the jps command, confirming successful execution of NameNode, DataNode, SecondaryNameNode, ResourceManager, and NodeManager services.

HDFS Commands Used

hdfs dfs -mkdir -p /retail/input

hdfs dfs -put retail_big.csv /retail/input/

hdfs dfs -ls -h /retail/input/

hdfs fsck /retail/input/retail_big.csv -files -blocks -locations

---

Hive Commands Used

CREATE DATABASE retail_bda;

USE retail_bda;

CREATE EXTERNAL TABLE retail_transactions (

transaction_id STRING,

customer_id STRING,

product_id STRING,

```
product_category STRING,
quantity INT,
unit_price DOUBLE,
transaction_date STRING,
store_id STRING,
region STRING,
payment_method STRING,
discount DOUBLE
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/retail/input'
TBLPROPERTIES ("skip.header.line.count"="1");
```

Main Hive Analytical Queries

Revenue by Product Category:

```
SELECT product_category,
ROUND(SUM(quantity * unit_price * (1 - discount)),2) AS total_revenue
FROM retail_transactions
GROUP BY product_category
ORDER BY total_revenue DESC;
```

Monthly Revenue:

```
SELECT SUBSTR(transaction_date,1,7) AS sales_month,
```

```
ROUND(SUM(quantity * unit_price * (1 - discount)),2) AS monthly_revenue
FROM retail_transactions
GROUP BY SUBSTR(transaction_date,1,7)
ORDER BY sales_month;
```

Regional Revenue:

```
SELECT region,
COUNT(*) AS transaction_count,
ROUND(SUM(quantity * unit_price * (1 - discount)),2) AS regional_revenue
FROM retail_transactions
GROUP BY region
ORDER BY regional_revenue DESC;
```

Top Customers:

```
SELECT customer_id,
ROUND(SUM(quantity * unit_price * (1 - discount)),2) AS customer_value
FROM retail_transactions
GROUP BY customer_id
ORDER BY customer_value DESC
LIMIT 10;
```

Hive Query Used for Revenue Analysis

```
SELECT
   product_category,
   ROUND(SUM(quantity * unit_price * (1 - discount)),2) AS total_revenue
FROM retail_transactions
GROUP BY product_category
```

ORDER BY total_revenue DESC;

Hive automatically converts this SQL statement into underlying MapReduce jobs executed through YARN, eliminating the need for manual Mapper and Reducer implementation.

Machine Learning Summary

Python libraries used:

- Pandas
- NumPy
- Scikit-learn
- Matplotlib

Machine Learning Model:

- Random Forest Regressor

Evaluation Metrics:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R^2 Score

Visualisations Generated:

- Revenue by Product Category
- Monthly Revenue Trend
- Revenue by Region
- Actual vs Predicted Revenue
- Feature Importance